Tarn

SDK Guide

Building apps on permanent, encrypted, user-owned data

May 2026

Contents

Tarn Client

App-facing SDK for Tarn — permanent, encrypted, user-owned data on Arweave.

The SDK is schema-first: you declare collections and fields up front, and the client gives you typed CRUD per collection plus typed namespaces for connections, sharing, the account key, account management, and sessions. Apps never touch Arweave txids, content-encryption keys, share-log seq numbers, or HPKE handshakes — those stay inside the SDK.

Ships full TypeScript declarations. Works in browsers and Node.js 18+.

Quick start

```
import { TarnClient, defineSchema, TarnStorage } from 'tarn-client';

const schema = defineSchema({
  appId: 'your-app',
  version: 1,
  collections: {
    notes: {
      primaryKey: 'noteId',
      fields: {
        noteId:   'string',
        title:    'string',
        body:     'string?',
        priority: 'integer?',
      },
      shareable: true,
    },
  },
});

const tarn = await TarnClient.create({
  apiBase: 'http://localhost:8787',
  appId:   'your-app',
  schema,
  storage: TarnStorage.memory(),
});

await tarn.register('me@example.com', 'p@ssw0rd', { recoveryAcknowledged: true });
// The first argument is the username (any UTF-8 string the app chooses — many
// apps populate it with an email address; Tarn does not require email format).

await tarn.notes.create({ noteId: 'n1', title: 'Hello, Tarn' });
console.log(await tarn.notes.list());
```

That's the whole shape. Everything below is detail.

Schema

A schema declares what collections exist, what fields each collection has, and which collections can be shared. `defineSchema()` brands the result so the client only accepts validated schemas, and validates the shape eagerly at module load.

```
import { defineSchema } from 'tarn-client';

export const schema = defineSchema({
  appId:   'your-app',
  version: 1,
  collections: {
    notes: {
      primaryKey: 'noteId',
      fields: {
        noteId:    'string',
        title:     'string',
        body:      'string?',
        priority:  { type: 'integer', required: false },
        updatedAt: 'date?',
        pinned:    'boolean?',
        status:    { type: 'string', enum: ['draft', 'active', 'archived'] },
      },
      shareable: true,
    },
    settings: {
      primaryKey: 'key',
      fields: {
        key:   'string',
        value: 'json',
      },
      // shareable defaults to false — share()/listShared() are not available.
    },
  },
});
```

Field types: `string`, `number`, `integer`, `boolean`, `date`, `json`. Append `?` for optional (`'string?'`), or use the long form `{ type, required, default, enum }`.

Reserved names: `cred`, `connection`, `share-log-state`, `share-inbox`, `recovery-factor`, `app-config`, `app-schema`. Declaring a collection with a reserved name throws at `defineSchema()`.

Sharing: only collections with `shareable: true` get `share/shareWithAll/unshare/listShared` methods. The schema is the authoritative answer to "is this thing shareable" — apps can't bypass it.

Validation: `tarn.<collection>.create()` and `update()` validate against the schema synchronously, before any network or crypto work. Missing required fields, wrong types, unknown fields, or enum violations throw `TarnSchemaError`.

Schema versioning and field evolution

Schemas carry a numeric `version`. Bumping it republishes the schema (via `tools/publish-schema.mjs`) under a new `v=` Arweave tag so the recovery client can pick the version that matches each record's `SchemaV` tag.

Three rules cover the common cases:

Change	Backward-compat?	What you do
Add a new optional field	yes	Bump `version`, republish. Older records read with the field absent; new writes include it.
Add a new required field with a default	yes	Bump `version`, republish. The default fills in for older records on read.
Add a new required field without a default	**no**	Older records fail validation on read. Either supply a default, or migrate (see below).
Remove a field	**no**	Older records still carry the field on disk; the strict validator rejects unknown fields on read. Either keep the field declared as deprecated (still accepted, no longer used), or migrate.
Rename a field	**no**	Same as remove + add. Migrate, don't rename in place.
Change a field's type	**no**	Always a breaking change. Migrate.
Tighten an enum (drop a value)	**no**	Older records carrying the dropped value fail. Don't drop; deprecate.
Loosen an enum (add a value)	yes	Older records still satisfy the union.

Migration pattern. Tarn doesn't ship a `migrate()` helper — apps write their own walk because the right semantics (one-shot vs. lazy, error handling, partial-failure recovery) are app-specific.

Cost matters. Every `update()` is a new permanent Arweave entry, paid via Turbo from the app's funder. Records aren't mutated in place — chained entries append a new version each time, and old versions stay on Arweave forever. A one-shot migration of N records on K users is N×K permanent paid writes. Small entries currently fall under Turbo's free tier, but that's pricing policy, not a guarantee.

Prefer schema changes that don't require migration. The first three rows of the table above — additive optional fields, additive required fields with defaults, loosened enums — cost zero writes. Reach for them first. Deprecate fields instead of removing them. Loosen enums instead of tightening. Most schema evolution can be designed to avoid migration entirely if you plan additively from the start.

When migration is unavoidable, the pattern is straightforward:

```
// One-shot migration: read all, transform, re-write under the new schema.
// Each update is a new permanent Arweave entry — budget accordingly.
const all = await tarn.notes.list();
for (const note of all) {
  if (note.body == null) {
    await tarn.notes.update(note.noteId, { body: '' });
  }
}
```

Validation runs on `update()`, so the migration loop fails fast if the transform is wrong. Re-running is safe — `update()` is idempotent on identical inputs.

Forward-looking — relaxed validation. The current validator is strict by design: unknown fields throw. A future option (`onUnknownField: 'strip' | 'preserve' | 'error'`) will let apps opt into laxer behaviour for upgrade paths — read with extra fields, log a warning, drop them on the next write. Default stays `'error'` so the schema-first commitment isn't watered down. Not shipped yet; track the issue if it matters for your migration plan.

Collections

Each `shareable: true` collection on the schema becomes a typed namespace on the client.

```
// Create. Validates against the schema. Returns the validated record.
await tarn.notes.create({ noteId: 'n1', title: 'Quarterly review', body: 'Pull metrics'
    });

// Get one. Returns null if no record matches.
const note = await tarn.notes.get('n1');

// List all live records.
const all = await tarn.notes.list();

// Partial update — SDK reads current, merges patch, writes a new entry.
await tarn.notes.update('n1', { priority: 5 });

// Tombstone.
```

```
await tarn.notes.delete('n1');
```

Records are addressed by **primary key**, never by Arweave txid. The SDK maps primary keys onto the protocol's `Eid` tag so reads converge across devices.

`update()` is **partial-merge**. Pass only what's changing; the SDK reads the current record from Arweave, merges, re-validates as a full record, and writes a chained entry. Full-replace is `update(id, { ...current, ...patch })` if you ever want it.

Sharing (when `shareable: true`)

```
const friends = await tarn.connections.list();

// Share one record with one friend.
await tarn.notes.share(friends[0], 'n1');

// Share with all connections (skips muted). Returns counts and per-conn failures.
const result = await tarn.notes.shareWithAll('n1');
// { ok: 3, failed: [{ connection, error }] }

// Revoke from one friend.
await tarn.notes.unshare(friends[0], 'n1');

// Read what a friend has shared with us under THIS collection.
const theirs = await tarn.notes.listShared(friends[0]);
```

Calling `share()` on a non-`shareable` collection throws — the schema is the gate.

Connections

`tarn.connections.*` covers the full lifecycle of friend connections.

```
// Username-based handshake (requires recipient to be a discoverable Tarn user).
await tarn.connections.invite('alice@example.com');

// Recipient lists incoming requests via the advanced surface, then accepts:
await tarn.connections.accept(requestNonce, { label: 'Alice — work team' });

// Listing.
const conns = await tarn.connections.list();
// [{ share_pub, signing_pub, label?, muted? }, ...]

// Mute / unmute. Mute is per-side, invisible to the other party.
await tarn.connections.mute(conns[0]);
await tarn.connections.unmute(conns[0]);
const muted = await tarn.connections.isMuted(conns[0]);

// Set or change a label after the fact.
await tarn.connections.setLabel(conns[0], 'Alice');

// Remove the connection. They stay around in your share-log history but
// new shares stop flowing.
```

```
await tarn.connections.remove(conns[0]);
```

Invite tokens — link / QR flow

For consumer apps where the inviter doesn't know the recipient's username (or the recipient isn't a Tarn user yet):

```
// Inviter — generate a single-use, time-limited link.
const { token_id, invite_url, expires_at } = await tarn.connections.createInvite({
  label:       'Maya',  // local-only: labels the connection that forms when this
    redeems
  expiry_days: 7,       // default 7, server max 30
});
// invite_url is something like:
//   https://app.example.com/invite/<token_id>#<base64url payload_key>
// Share it via QR / messenger / email / etc.

// Recipient — extract token_id from path, payload_key from URL fragment.
const tokenId    = new URL(location.href).pathname.split('/').pop();
const payloadKey = location.hash.slice(1);
await tarn.connections.redeemInvite(tokenId, payloadKey);
// The inviter's SDK auto-accepts the resulting connection request on next poll.
```

The URL fragment (`#`) is never transmitted to the API — the payload key stays on the recipient's device. Tarn stores opaque ciphertext keyed on `token_id` and never sees the inviter's identity.

`label` is local: it's stored only in the inviter's encrypted `tarn-issued-invites-v1` record, surfaced in `listIssuedInvites()` , and used to seed `Connection.label` when the matching redemption auto-accepts. The recipient never sees it. If your app wants to show the recipient who's reaching out (e.g. "Maya invited you"), pass that name through your delivery channel — the message body of the email/SMS/QR-page that carries the invite URL.

Account key

Tarn issues every account a 24-word BIP39 account key at signup. The account key is a parallel access path to the user's data — independent of the password. Apps **must** surface the account key to the user during registration; passing `recoveryAcknowledged: true` is the SDK's way of forcing the conversation.

```
const reg = await tarn.register('me@example.com', 'p@ssw0rd', {
  // First arg is the username. See "Authentication and the username field" below.
  recoveryAcknowledged: true,
  // storeAccountKey defaults to true (Model B). Pass false for strict
  // zero-knowledge "no backup stored" registration (Model A).
  storeAccountKey: true,
});
// reg.accountKey       — 24-word string
// Hand it to the user immediately. Do NOT persist it.
```

Kit format and delivery are the app's job. Tarn returns the 24-word account-key string and nothing else — no PDF, no rendered bytes. Apps render their own kit (downloadable PDF,

printable HTML, clipboard copy, whatever fits) and decide how to surface it to the user. There is no Tarn endpoint that handles plaintext account-key material, even ephemerally; the SDK does not ship an in-bundle PDF renderer either, so apps stay in full control of branding and layout. If the application wants to email the kit, it must operate the transport itself — Tarn will not host a forwarder, since routing account-key material through Tarn-operated infrastructure would weaken the zero-knowledge guarantee that applies to everything else in the protocol.

Storage models — Model A vs Model B

Apps pick the storage posture at registration via the `storeAccountKey` option:

- `storeAccountKey: true` **(default, Model B).** The SDK encrypts the account key under the user's gen-1 DEK with AAD `"tarn-wrapped-account-key-v1"` and ships the ciphertext to Tarn. Logged-in users can later retrieve and view the account key from app settings via `tarn.accountKey.view()`. The wrap is opaque to Tarn; only a user who supplies their password can decrypt it.
- `storeAccountKey: false` **(Model A).** The wrap is omitted entirely. Tarn never holds any form of the account key. Lose the password AND the account key → data is permanently inaccessible (even Tarn cannot help). This is the strict zero-knowledge posture.

Reading the storage state at runtime:

```
// After login or register completes:
tarn.accountKey.isStored();   // true (Model B) | false (Model A) | null (no auth round
    trip yet)
```

Viewing the account key (Model B)

```
const { accountKey } = await tarn.accountKey.view({ password:
    'freshly-re-entered-password' });
// 24-word string. Surface it briefly; do not persist.
```

The flow:

1. The SDK runs a step-up auth dance against the API: fresh nonce → sign with credential signing key derived from the re-entered password → exchange for a single-use 60-second token.
2. The SDK fetches the encrypted wrap with both the session JWT and the step-up token.
3. The SDK decrypts the wrap client-side and runs a wrap-pinning check (re-derives the `recovery_lookup_key` from the decrypted phrase and compares to the server-stored value). On mismatch, `view()` throws `AccountKeyPinningError` — surface this distinctly from a "wrong password" or "network error", since it's a security warning.

Failure modes:

- **Wrong password** — step-up authentication fails. The SDK throws an `Error` whose message contains `"step-up"` or `"challenge"`.
- **Model A account** — the server returns 404. The SDK throws an `Error` whose message contains `"no_account_key_stored"`. Render the appropriate "no backup stored" UI.

- **Pinning mismatch** — the SDK throws `AccountKeyPinningError`. Treat as a security warning and avoid surfacing the (would-be) phrase.
- **Network / decryption failure** — propagated as a regular `Error`.

Toggling storage at runtime

Users can switch their storage posture at any time after registration. Both methods drive a step-up auth dance internally — the caller passes the freshly-entered password.

```
// Model A → Model B. Caller must also supply the user's existing
// account key (the SDK does not retain it past register / view / rotate).
// Apps prompt the user to type it, validate via validateAccountKey(),
// then pass it here.
await tarn.accountKey.enableKeyStorage({
  password:   'freshly-re-entered-password',
  accountKey: '24 words ...',
});

// Model B → Model A. Just the password — no phrase needed.
await tarn.accountKey.disableKeyStorage({ password: 'freshly-re-entered-password' });
```

`enableKeyStorage` runs a wrap-pinning check before submitting: it derives `recovery_lookup_key` from the supplied phrase and compares to the value cached on the client. If the user types a valid 24-word phrase that doesn't actually belong to this account, the SDK throws `AccountKeyPinningError` BEFORE any server round trip. The pin check requires the SDK to have learned `recovery_lookup_key` (set at register / recoverAccount); on a login-only session the value isn't available and the pin check is skipped — the server is the only line of defense in that case, which is fine.

`disableKeyStorage` is idempotent: calling it on an already-Model-A account returns `{ stored: false, alreadyDisabled: true }` instead of throwing. UI code can treat both responses identically.

Failure modes for both:

- **Wrong password** — step-up fails. Error message contains `"step-up auth failed"`.
- **Pin-check failure** (`enableKeyStorage` only) — `AccountKeyPinningError`.
- **Network / D1 failure** — propagated as a regular `Error`.

Rotating the account key

When the user suspects their existing account key has been compromised (or wants to evict it as a routine hygiene step), `rotate()` generates a fresh one, re-wraps the entire DEK chain under the new recovery factor, and atomically publishes the bundle. The OLD account key stops working for `recoverAccount` immediately after this returns.

```
const { accountKey: newPhrase } = await tarn.accountKey.rotate({
  password: 'current-password',
});
// Surface newPhrase to the user briefly (downloadable kit, printable
// page, etc.). The SDK does not retain it.
```

Notes:

- The user's password and username are NOT changed by rotation — only the account-key half of the recovery factor.
- The recovery salt is regenerated as part of the new envelope (clean reset of the recovery state).
- If the account is in Model B, the new account key is also stored (re-wrapped under the existing gen-1 DEK). Model A stays Model A.
- Pre-rotation data remains decryptable — the DEK chain itself is unchanged; only the wrappings rotated.
- Apps can also pass `rotatePhrase: true` to `tarn.recoverAccount()` to bundle a rotation into a forgot-password flow (see below).

Failure modes:

- **Wrong password** — local credential mismatch tripwire. Error message contains `"wrong password"`.
- **Conflict** — `409` from the API if the new `recovery_lookup_key` collides with another account (astronomically unlikely; retry yields a fresh key). Error message contains `"conflict"`.
- **Network / D1 failure** — propagated as a regular `Error`. The D1 update is atomic, so partial state cannot occur; safe to retry.

Account recovery (when the password is lost)

Account recovery itself goes through the top-level `tarn.recoverAccount()` (auth lifecycle):

```
await tarn.recoverAccount({
  phrase:      '24 words ...',
  newUsername: 'me@example.com',
  newPassword: 'fresh-password',
});
// New credentials re-wrap the existing data keys; all pre-recovery data is decryptable.
```

Optional `{ rotatePhrase: true }`**.** Pass this to bundle an account-key rotation into the recovery flow — useful when the user is recovering BECAUSE they suspect the phrase itself has been compromised (e.g. "I think someone has my recovery sheet"):

```
const result = await tarn.recoverAccount({
  phrase:       '24 words ...',
  newUsername:  'me@example.com',
  newPassword:  'fresh-password',
  rotatePhrase: true,
});
// result.accountKey contains the NEW phrase; surface it to the user
// once and let them save it. The OLD phrase no longer authenticates.
```

When `rotatePhrase` is omitted (default `false`), recovery preserves the existing phrase — exactly as documented in the protocol. The result has no `accountKey` field. If the inline rotation

fails after a successful recovery, the SDK throws a partial-success error pointing at `tarn.accountKey.rotate()` for retry.

Authentication and the username field

Tarn does not validate the username's format. It's a UTF-8 string used as a KDF salt input (after `trim().toLowerCase()` normalization) and as a public user-lookup key for connection bootstrap. Apps choose whether to enforce email-shape, handle-shape, phone-shape, or anything else; the SDK neutrally calls the parameter `username`. Many apps will populate it with an email address — that's fine and expected — but Tarn does not require this.

Account + Session

```
// Rotate credentials (routine username/password change). Existing data stays
    decryptable.
// Pass the account key to extend the recovery factor to the new generation.
await tarn.account.changeCredentials('new@example.com', 'new-password', { phrase });

// Permanently delete. Tombstones credentials, clears server-side state, wipes local
    session.
await tarn.account.delete();

// Local session.
tarn.session.isLoggedIn();          // boolean — whether keys are loaded
await tarn.session.clear();         // forget local session; user must re-auth

// Server-side session management (one row per device).
const devices = await tarn.session.listDevices();
// [{ sid, createdAt, lastSeenAt, deviceLabel, viaRecovery, isCurrent }, ...]

await tarn.session.revokeDevice(devices[1].sid);   // log one device out
await tarn.session.revokeAllOthers();              // "log out everywhere else"
await tarn.session.revokeAll();                    // log out everywhere including here
```

Apps **should** attach a device label at login time — `tarn.login(username, password, { deviceLabel: 'Chrome on MacBook' })` — so users can identify their sessions in `listDevices()`. If omitted, `deviceLabel` is `null` and users only see the opaque `sid` and timestamps. Pick something the user will recognize: a User-Agent summary, a hostname, or a name the user typed at signup.

Advanced (escape hatches)

Power-user surface for prototyping, debugging, and use cases the typed namespaces don't cover. Most apps never need this.

```
// Schema-less entry CRUD — bypasses the collection layer entirely.
await tarn.advanced.entries.create('arbitrary-type', { foo: 'bar' });
await tarn.advanced.entries.update(priorTxid, 'arbitrary-type', { foo: 'baz' });
```

```
await tarn.advanced.entries.delete(targetTxid, 'arbitrary-type');

// Raw blob fetch + decrypt by shareKey. Useful for one-off recipient flows.
const blob = await tarn.advanced.entries.fetchBlob(txid);
const data = await tarn.advanced.entries.decryptSharedBlob(blob, shareKey);

// Direct share-log access. Collection.share()/listShared() are the typed wrappers.
await tarn.advanced.shareLog.read(connection);
await tarn.advanced.shareLog.share(connection, contentId, txid, shareKey);
await tarn.advanced.shareLog.unshare(connection, contentId);
```

If you find yourself reaching for `advanced.*` for something the typed surface should cover, that's a signal to file an issue.

App registration

Before users can register on your app, the app itself must be registered with Tarn. This is operator-level setup; it happens once per app, not per user.

1. Generate an app key pair

```
node tools/generate-app-key.mjs your-app-id
```

Prints the private key (hex), the public key, and a D1 seed SQL line. **Save the private key in a password manager** — it's printed once.

2. Register the app in Tarn

Run the printed D1 seed command:

```
# Local
cd api && npx wrangler d1 execute tarn-api --local --command "<printed SQL>"
# Production
cd api && npx wrangler d1 execute tarn-api --remote --command "<printed SQL>"
```

3. Set per-account rules

After a user registers, set their write rules:

```
node tools/set-rules.mjs \
  --api  https://api.tarn.dev \
  --app  your-app-id \
  --key  <private key hex> \
  --dlk  <user's data_lookup_key> \
  --plan free        # or annual, clear, deny, or --rules '<JSON>'
```

Plan / type	Effect
`free`	5 entries, 100KB max per entry
`annual`	1000 entries, expires in 1 year

Plan / type	Effect
`clear`	Empty rules (allow all)
`deny`	Deny all writes
`max_entries`	`{ limit, since?, app?, entry_type? }`
`max_bytes`	`{ limit }` per entry
`expires`	`{ at }` ISO 8601

Rules are AND logic; unknown rule types fail closed.

4. Publish the schema

The schema is published to Arweave so the future "always access your data" recovery client can read it without going through the Tarn API. Run this once per release that bumps `schema.version`:

```
node tools/publish-schema.mjs \
  --api    https://api.tarn.dev \
  --app    your-app-id \
  --key    <private key hex> \
  --schema ./schema.mjs
```

`--schema` accepts either a `.json` file (serialized schema) or a `.mjs` file with a default export equal to the schema. Re-running with the same `(app, version, schema)` is a safe no-op.

Examples

Four progressive examples live in `../examples/`:

- `01-hello-world` — register, write one record, list it.
- `02-crud` — full CRUD on two collections.
- `03-sharing` — invite-token handshake + share-with-all flow.
- `04-recovery` — register, export the account-key PDF, simulate password loss + recoverAccount.

Each example is a standalone npm package linking to this client via `file:../../client`. See `examples/README.md` for setup.

TypeScript

The SDK is written in TypeScript and ships full `.d.ts` declarations. Schema-derived types flow through the client:

```
const tarn = await TarnClient.create({ schema: mySchema, /* ... */ });

await tarn.notes.create({
  noteId: 'n1',
  title:  'Foo',
  // titel: 'typo'    // ✗ TS error: unknown field
});

const note = await tarn.notes.get('n1');
//    ^? { noteId: string; title: string; body?: string; priority?: number; ... }
```

Plain JavaScript works too — the inference simply doesn't run. The runtime validators still enforce the schema at write time.

Security model

- **Client-side encryption.** All data is AES-256-GCM encrypted before leaving the client. The server never sees plaintext.
- **Argon2id KDF.** Memory-hard (m=64 MiB, t=3, p=1) for password→master-key. Sub-keys derived via HKDF-Expand.
- **ECDSA P-256 auth.** Challenge-response signing. No passwords transmitted; server stores only the public key.
- **Per-content CEK + forward-secret DEK rotation.** Each blob is encrypted with its own random CEK, wrapped under a generation-indexed DEK chain. Credential changes append a fresh DEK so post-rotation writes are unreachable from old credentials.
- **Multi-factor DEK chain.** Each chain entry is wrapped twice — once under a password-derived KEK, once under an account-key-derived KEK. Either factor independently unwraps the chain.
- **Arweave permanence.** Data is stored permanently on Arweave. Encrypted blobs are publicly visible but unreadable without the key.

Publicly observable metadata

Tarn protects content end-to-end, but a few metadata properties remain visible:

- **Connection-request inbox volume + timing** is observable to anyone who knows a user's `share_pub`. Casual social use is fine; sensitive contexts should consider `share_discoverable: false`.
- **Account existence via discoverability lookup** — a `share_discoverable: true` account leaks "this username is a Tarn user" to anyone who runs the lookup.
- **Once connected, share-log traffic is unlinkable** — per-pair tags are stealth-addressed; an Arweave observer cannot extract the connection graph from the protocol alone.

See TARN_PROTOCOL.md § Publicly observable metadata for the full breakdown, and TARN_PROTOCOL.md for the wire-protocol spec.